

DECOLAGEM DA MISSÃO

O SURGIMENTO DA INTELIGÊNCIA QUE APOIA A TOMADA DE DECISÃO

6

LISTA DE FIGURAS

Figura 1 – <i>Timeline</i> da IA	6
Figura 2 – Aprendizado supervisionado	12
Figura 3 – Aprendizado não supervisionado	13
Figura 4 – Aprendizado por reforço	15
Figura 5 – Regressão linear	18
Figura 6 – Classificação <i>versus</i> regressão linear	20
Figura 7 – <i>Clustering versus</i> classificação	21
Figura 8 – <i>Pipeline</i> de projetos de <i>machine learning</i>	22
Figura 9 – Validação cruzada <i>K-fold</i>	24
Figura 10 – Método do cotovelo	27
Figura 11 – <i>Perceptron</i> , neurônio artificial	30
Figura 12 – <i>Perceptron</i> de múltiplas camadas	32
Figura 13 – MLP vs <i>deep learning</i>	36
Figura 14 – <i>Machine learning versus deep learning</i>	37

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Classificação em Python.....	40
Código-fonte 2 – Regressão em Python	41
Código-fonte 3 – Clusterização em Python	41
Código-fonte 4 – Redes neurais em Python.....	42

EMSE

SUMÁRIO

FUNDAMENTOS DE IA	5
1 INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL	5
1.1 O que é IA? – História e evolução	5
1.2 Por que IA agora?	7
1.3 IA no mundo real: Aplicações por setor	8
1.4 Como pensar como um cientista de dados ou engenheiro de IA	9
2 TIPOS DE APRENDIZADO DE MÁQUINA	10
2.1 Aprendizado supervisionado	10
2.2 Aprendizado não supervisionado	12
2.3 Aprendizado por reforço	13
2.4 Comparação entre os tipos de aprendizado	15
3 CLASSIFICAÇÃO, REGRESSÃO E CLUSTERING	16
3.1 Regressão	16
3.2 Classificação	18
3.3 <i>Clustering</i>	20
3.4 Considerações práticas	21
4 VALIDAÇÃO E AVALIAÇÃO DE MODELOS	23
4.1 Divisão de dados	23
4.2 Métricas de regressão	25
4.3 Métricas de classificação	26
4.4 Métricas para <i>clustering</i>	27
4.5 Comparando modelos e escolhendo o melhor	28
5 REDES NEURAIS ARTIFICIAIS	28
5.1 Redes neurais: Intuição	29
5.2 Arquiteturas	30
5.3 Treinamento	32
5.4 Regularização	33
6 DEEP LEARNING	34
6.1 O que significa “profundo”?	35
6.2 Por que <i>deep learning</i> funciona tão bem?	36
6.3 Arquiteturas	37
6.4 Desafios do <i>deep learning</i>	38
7 LABORATÓRIO HANDS ON	39
7.1 Classificação	40
7.2 Regressão	40
7.3 Clusterização	41
7.4 Redes neurais	41
8 CONCLUSÃO	42
REFERÊNCIAS	44

FUNDAMENTOS DE IA

1 INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL

1.1 O que é IA? – História e evolução

A inteligência artificial, como campo de estudo, pode ser entendida como o esforço científico e tecnológico de criar máquinas capazes de executar tarefas que, até pouco tempo atrás, dependiam exclusivamente da inteligência humana. Embora o termo “IA” tenha se popularizado recentemente com o surgimento de modelos avançados como os *Large Language Models*, sua origem remonta a ideias formuladas muito antes da era moderna dos computadores. Entender essa trajetória é essencial para compreender por que a IA se tornou o eixo tecnológico da década e como ela evoluiu de um conjunto de hipóteses filosóficas para uma disciplina robusta que transforma a sociedade em escala global.

A primeira semente conceitual surgiu com Alan Turing, matemático britânico considerado um dos pais da computação. Em 1950, Turing publicou o artigo “Computing Machinery and Intelligence”, no qual propôs a famosa pergunta “as máquinas podem pensar?”. Em vez de responder diretamente, ele sugeriu um teste prático, hoje conhecido como teste de Turing, em que uma máquina tentaria imitar o comportamento humano ao ponto de enganar um avaliador. Esse teste inaugurou uma nova maneira de pensar sobre inteligência artificial: não mais como uma propriedade metafísica, mas como algo mensurável e operacional.

Poucos anos depois, em 1956, no Dartmouth College, nos Estados Unidos, ocorreu a conferência que marcou oficialmente o nascimento do campo de IA. John McCarthy, Marvin Minsky, Claude Shannon e outros pesquisadores reuniram-se com a ambiciosa hipótese de que a inteligência humana pode ser tão sistematicamente descrita que uma máquina seria capaz de simulá-la. Essa conferência, conhecida simplesmente como “Dartmouth Conference”, foi um divisor de águas: estabeleceu as bases conceituais da área, criou seu vocabulário inicial e consolidou o termo *artificial intelligence*.

O Surgimento da Inteligência que Apoia a Tomada de Decisão

Durante as décadas de 1950 e 1960, o campo viveu seu primeiro grande boom. Pesquisadores desenvolveram sistemas capazes de resolver problemas de álgebra, jogar damas e manipular linguagem de maneira básica. Havia uma confiança quase ilimitada de que, em poucos anos, máquinas inteligentes superariam seres humanos em muitas tarefas cognitivas. Contudo, essa empolgação encontrou rapidamente limites concretos. A falta de poder computacional, os algoritmos pouco eficientes e a dificuldade de lidar com problemas do mundo real provocaram frustração entre cientistas e órgãos de fomento.

Esse período, entre o fim da década de 1970 e meados da década de 1990, ficou conhecido como o “Inverno da IA”. Durante esse intervalo, investimentos foram cortados, pesquisas desaceleraram e a percepção pública tornou-se cética. Entretanto, mesmo com o cenário desfavorável, avanços fundamentais surgiram, como o desenvolvimento de redes neurais multicamadas e a redescoberta do algoritmo de *backpropagation*.

Com a virada do século, a área renasceu impulsionada pelo crescimento exponencial de dados digitais, pelo aumento do poder computacional e pela disponibilidade de algoritmos mais eficientes. Esse renascimento culminou na era do *deep learning* a partir de 2012, quando redes neurais profundas passaram a superar todos os métodos tradicionais em visão computacional, processamento de linguagem natural e várias outras tarefas. Esse momento histórico redefiniu completamente a IA e abriu caminho para os sistemas modernos que utilizamos atualmente em aplicações industriais, médicas, financeiras e científicas.



Figura 1 – *Timeline* da IA
Fonte: Ideias esquecidas (2023)

1.2 Por que IA agora?

Embora muitos dos conceitos fundamentais da IA já existissem há décadas, foi apenas recentemente que o campo encontrou o conjunto ideal de condições para florescer. A primeira dessas circunstâncias é o fenômeno do *big data*. Nas últimas duas décadas, o mundo digital produziu uma quantidade de dados sem precedentes. Redes sociais, sensores industriais, compras online, dispositivos móveis, câmeras de vigilância e serviços de *streaming* geram continuamente trilhões de dados que podem ser transformados em padrões úteis para sistemas inteligentes. A IA moderna depende intensamente de dados para aprender, e nunca houve um ambiente tão propício quanto o atual.

Outra condição essencial para o sucesso contemporâneo da IA foi a evolução do poder computacional. GPU's (*Graphics Processing Unit*), originalmente projetadas para processar gráficos em videogames, tornaram-se ferramentas extraordinariamente eficientes para treinar redes neurais devido à sua capacidade de realizar operações paralelas em larga escala. Com o tempo, surgiram ainda as TPU's (*Tensor Processing Units*), unidades especializadas criadas pelo Google especificamente para acelerar o treinamento de modelos de aprendizado profundo. Essas inovações tornaram possível treinar modelos antes considerados inviáveis devido ao custo computacional.

O movimento *open-source* também desempenhou um papel fundamental. Bibliotecas como *TensorFlow*, *PyTorch*, *Scikit-learn*, *Hugging Face Transformers* e outras democratizaram o acesso a algoritmos avançados de IA. Pesquisadores, estudantes e empresas podem experimentar, adaptar e criar modelos complexos sem a necessidade de infraestrutura proprietária. Esse espírito colaborativo acelerou exponencialmente o avanço da área e reduziu barreiras que antes limitavam a inovação.

Por fim, a convergência tecnológica transformou a IA em uma força dominante. A combinação de *cloud computing*, dispositivos móveis, internet de alta velocidade, sensores IoT, computação embarcada e sistemas distribuídos criou um ecossistema ideal para a implantação de IA em escala global. A computação moderna é um ambiente onde dados, processamento, modelos pré-treinados e integração entre

sistemas coexistem, permitindo que soluções inteligentes sejam implementadas de forma contínua, em tempo real e em praticamente qualquer domínio.

1.3 IA no mundo real: Aplicações por setor

A inteligência artificial não é apenas uma tecnologia promissora; ela já desempenha papel central em inúmeros setores da economia e da vida cotidiana. Na área da saúde, algoritmos de aprendizado supervisionado e redes neurais são usados para detectar tumores em exames de imagem com precisão comparável ou superior à de especialistas humanos, além de prever riscos clínicos, personalizar tratamentos e otimizar o fluxo hospitalar. Em muitos hospitais, sistemas baseados em IA ajudam médicos a identificarem padrões sutis que seriam impossíveis de detectar manualmente, acelerando diagnósticos e aumentando a segurança do paciente.

No setor de varejo, a IA é empregada para prever demanda, detectar ruptura de estoque, ajustar preços dinamicamente, recomendar produtos e aprimorar a experiência do cliente. Grandes redes utilizam modelos que analisam milhões de transações para antecipar o comportamento de consumidores, otimizar logística e reduzir custos operacionais. A IA tornou-se uma ferramenta indispensável para competir em mercados altamente voláteis.

Na indústria 4.0, algoritmos analisam sinais de máquinas para prever falhas antes que ocorram, permitindo manutenção preditiva e reduzindo paralisações inesperadas. Robôs inteligentes colaboram com seres humanos em linhas de produção, enquanto sistemas de visão computacional monitoram qualidade em tempo real com precisão superior à inspeção manual. A combinação de IA, sensores e automação transformou fábricas inteiras em ambientes autônomos e adaptativos.

No setor financeiro, modelos de IA detectam fraudes em transações, avaliam risco de crédito, automatizam investimentos e auxiliam na conformidade regulatória. Os sistemas de *trading* algorítmico analisam grandes volumes de dados para tomar decisões em milissegundos. A IA tornou-se tão central que praticamente todas as instituições financeiras globais dependem dela para operar com eficiência.

Na educação, sistemas inteligentes personalizam conteúdos para cada estudante, identificam dificuldades precoces, recomendam trilhas de aprendizagem e auxiliam professores na avaliação. Os LLMs (*Large Language Model*) ampliaram

ainda mais esse impacto, oferecendo tutores virtuais, correção de redações, explicações passo a passo e ferramentas de apoio ao ensino.

Esses exemplos ilustram como a IA se tornou uma tecnologia horizontal. Ela não pertence mais a um setor específico, mas permeia todas as áreas da sociedade, funcionando como uma camada fundamental que potencializa operações, decisões e inovações em qualquer contexto.

1.4 Como pensar como um cientista de dados ou engenheiro de IA

Compreender os fundamentos da IA não significa apenas conhecer algoritmos, mas adotar uma forma específica de raciocínio. Um cientista de dados ou engenheiro de IA precisa ser capaz de traduzir problemas do mundo real em problemas computacionais solucionáveis. Esse processo envolve analisar o fenômeno em questão, determinar quais dados são necessários, definir qual abordagem de aprendizado é adequada e compreender como avaliar se uma solução é suficientemente boa.

O fluxo geral de um projeto de IA costuma seguir uma sequência lógica. Inicialmente, é necessário definir claramente o problema e seu objetivo. Muitas vezes, essa etapa exige discutir com especialistas de domínio, clientes ou *stakeholders* para entender o contexto operacional e quais decisões precisam ser tomadas. Depois, inicia-se o processo de coleta e limpeza de dados, que geralmente é a etapa mais trabalhosa. A qualidade dos dados influencia diretamente a qualidade do modelo, e por isso profissionais de IA desenvolvem um olhar crítico para detectar inconsistências, erros, *outliers* e padrões suspeitos.

Uma vez que os dados estejam preparados, o próximo passo é selecionar o tipo de aprendizado adequado: supervisionado, não supervisionado ou por reforço, e escolher algoritmos que sejam compatíveis com a estrutura do problema. Em seguida, modelos são treinados, validados e ajustados. Essa etapa raramente é linear: requer iteração, experimentação e comparação entre diferentes alternativas.

Finalmente, a solução precisa ser interpretada, validada com métricas adequadas e integrada ao ambiente real. Essa integração frequentemente envolve arquitetura de sistemas, API's, bases de dados, governança de modelos e

monitoramento contínuo para garantir que o desempenho permaneça estável ao longo do tempo.

Pensar como um profissional de IA é, portanto, um exercício de raciocínio analítico, pensamento estatístico, domínio técnico e, acima de tudo, capacidade de transformar perguntas abertas em problemas matematicamente estruturados.

2 TIPOS DE APRENDIZADO DE MÁQUINA

O aprendizado de máquina é o núcleo da inteligência artificial moderna. Ele permite que sistemas automatizados aprendam padrões, reconheçam regularidades, tomem decisões e façam previsões com base em dados. A ideia central é fornecer aos algoritmos exemplos suficientes para que eles possam extrair estruturas e relações escondidas, imitando, até certo ponto, a capacidade humana de aprender com experiência. O aprendizado de máquina é tradicionalmente dividido em três tipos principais: o supervisionado, o não supervisionado e o por reforço. Cada uma dessas abordagens resolve problemas diferentes, utiliza estruturas distintas de dados e envolve modelos matemáticos próprios.

2.1 Aprendizado supervisionado

O aprendizado supervisionado é a forma mais comum e estudada de aprendizado de máquina. Ele consiste em treinar um modelo utilizando exemplos rotulados, ou seja, pares de entrada e saída conhecidos. Cada exemplo é definido como um par (x, y) , em que x representa as características observáveis do fenômeno estudado e y representa a resposta ou rótulo correto associado àquela entrada. O objetivo do modelo é aprender uma função f capaz de mapear as entradas para as saídas com o menor erro possível. Depois que essa função é aprendida, o modelo pode generalizar e prever a saída de novas entradas nunca vistas.

Para entender esse processo, imagine um problema de previsão de demanda em um supermercado. Todos os dias, para cada produto, coletam-se dados como vendas passadas, preço atual, dia da semana, eventos sazonais e presença de promoções. Essa coleção de variáveis forma o x . Já o y seria o número real de unidades vendidas em um dia específico. O modelo aprende, a partir de centenas de

milhares de exemplos históricos, quais padrões relacionam essas variáveis ao resultado. Uma vez treinado, ele pode prever quantas unidades provavelmente serão vendidas amanhã, permitindo que gestores tomem decisões sobre compra, estoque e logística.

Outro exemplo clássico é o diagnóstico clínico assistido por IA. Nesse caso, o x pode representar uma imagem médica, como um raio-X ou uma tomografia, enquanto o y seria um rótulo indicando a presença ou ausência de uma condição específica, por exemplo pneumonia. O modelo aprende, a partir de milhares de imagens enviadas por especialistas, a identificar regiões opacas, densidades anormais e padrões que são difíceis de detectar manualmente. O resultado é um sistema capaz de auxiliar médicos, aumentando precisão diagnóstica e reduzindo tempo de análise.

O cerne do aprendizado supervisionado envolve minimizar uma função de perda, também chamada de função custo. Essa função mede o quão distante a predição do modelo está do valor real. Se chamarmos a previsão do modelo de $f(x)$ e o rótulo verdadeiro de y , então a função de perda $L(y, f(x))$ quantifica o erro para aquele exemplo. A versão agregada sobre todos os dados é chamada de risco empírico, que representa a média da perda calculada em todos os exemplos de treinamento. Formalmente, podemos expressar esse conceito como:

$$\text{Risco empírico} = \frac{1}{n} \sum L(y_i, f(x_i))$$

O objetivo do treinamento é ajustar os parâmetros internos do modelo para minimizar esse risco empírico. Esse processo geralmente é realizado por algoritmos de otimização, como o gradiente descendente, que ajustam os parâmetros de forma iterativa até alcançar um ponto em que a perda seja suficientemente pequena.

O fluxo geral do aprendizado supervisionado pode ser visualizado como um esquema contínuo que se inicia com dados rotulados, avança para um processo de treinamento e culmina em um modelo capaz de realizar previsões.

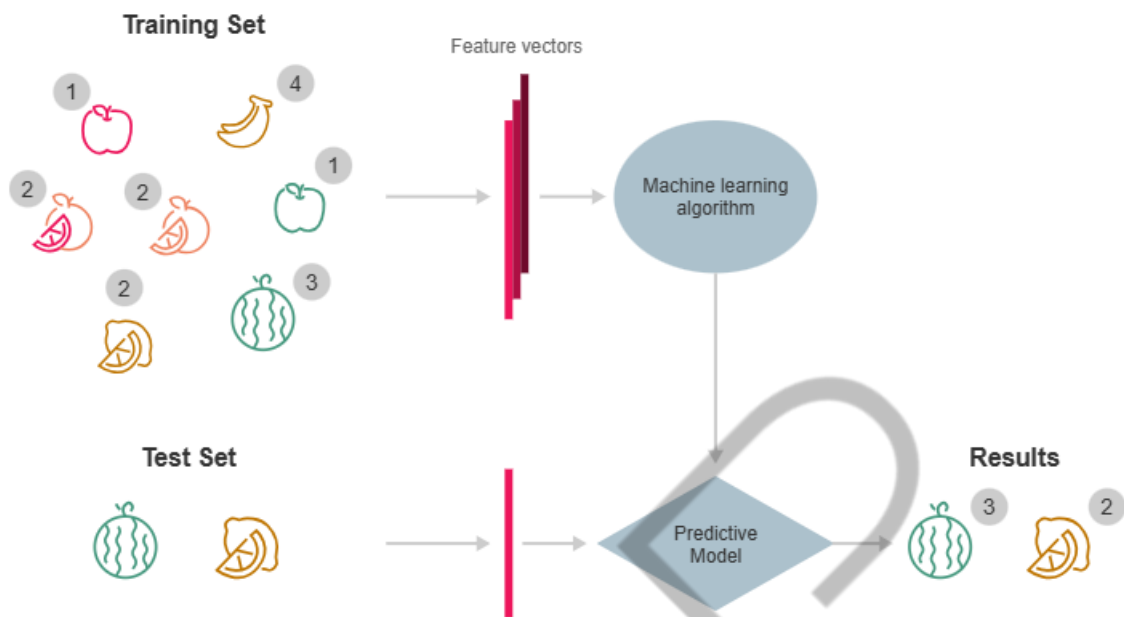


Figura 2 – Aprendizado supervisionado
Fonte: Shift Asia (2025)

2.2 Aprendizado não supervisionado

Enquanto o aprendizado supervisionado depende de rótulos fornecidos por especialistas, o aprendizado não supervisionado funciona de forma completamente diferente. Aqui, o modelo recebe apenas dados brutos, sem qualquer informação explícita sobre qual deve ser o resultado final. Sua tarefa é encontrar estruturas escondidas, padrões recorrentes e relações subjacentes nos dados. Esse tipo de aprendizado é extremamente valioso quando os rótulos são caros, subjetivos, difíceis de definir ou simplesmente inexistentes.

Um dos usos mais comuns do aprendizado não supervisionado é a detecção de padrões e agrupamentos. Nesse contexto, algoritmos como *K-Means* procuram dividir os dados em grupos que compartilham características semelhantes. Imagine um grande supermercado com milhões de registros de compra. Cada cliente pode ser representado por um conjunto de variáveis que incluem frequência de visitas, ticket médio, categorias preferidas e sensibilidade a promoções. A partir desses dados, algoritmos de *clustering* podem identificar grupos naturais de clientes, como compradores ocasionais, clientes fidelizados e consumidores de alto valor. Essa segmentação é poderosa porque permite personalizar ações de marketing e melhorar estratégias de retenção.

Outro uso importante é a redução de dimensionalidade. Em muitos problemas, especialmente aqueles envolvendo imagens, genômica, sensores industriais ou dados financeiros, as variáveis podem chegar a centenas ou milhares. Isso torna o processamento caro e dificulta a visualização. Técnicas como PCA (*Principal Component Analysis*) transformam os dados em novas representações que preservam a maior parte da variabilidade, mas com bem menos dimensões. Esse processo facilita a análise, acelera modelos supervisionados e elimina ruídos que prejudicam o desempenho.

Aplicações práticas do aprendizado não supervisionado surgem constantemente em áreas como detecção de anomalias, em que o modelo aprende o comportamento "normal" dos dados e identifica automaticamente qualquer padrão que se desvia do esperado. Essa técnica é usada em bancos para identificar transações suspeitas, em fábricas para detectar vibrações anormais em máquinas e em sistemas de segurança para reconhecer acessos incomuns em redes corporativas.

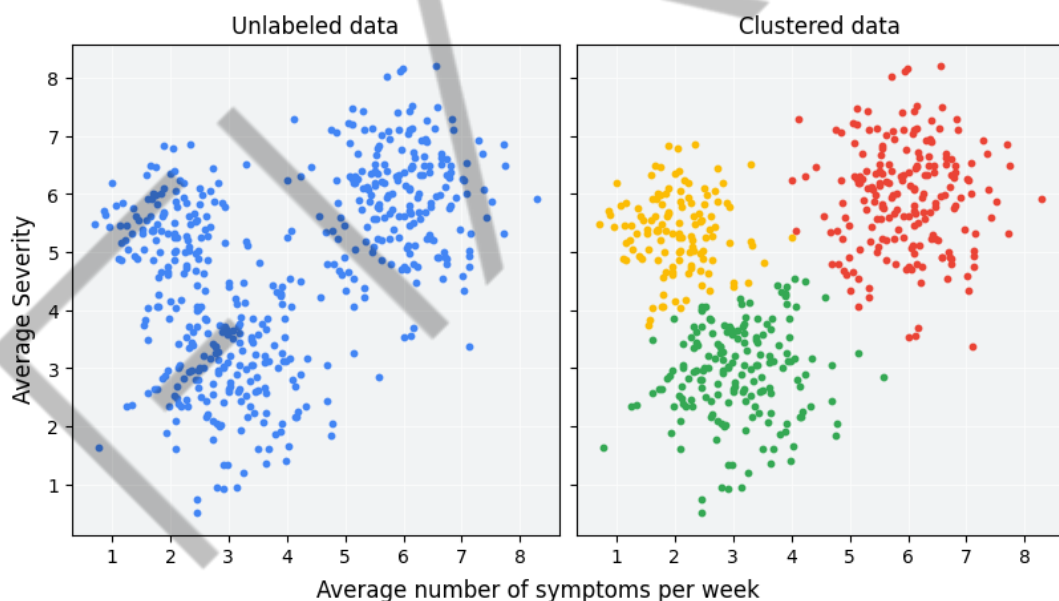


Figura 3 – Aprendizado não supervisionado
Fonte: Google Developers (2025)

2.3 Aprendizado por reforço

O aprendizado por reforço é uma abordagem diferente das duas anteriores. Ele se baseia no conceito de agentes que interagem com um ambiente, tomam ações e recebem recompensas com base no resultado dessas ações. A tarefa do agente é aprender uma política ótima, ou seja, um conjunto de regras para decidir qual ação

tomar em cada estado do ambiente, maximizando a recompensa acumulada ao longo do tempo. Esse tipo de aprendizado é inspirado na psicologia comportamental e nos mecanismos de aprendizagem presentes em animais e seres humanos.

No aprendizado por reforço, há três elementos principais: o primeiro é o agente, que é o sistema inteligente que toma decisões; o segundo é o ambiente, que responde às ações do agente de maneira previsível ou imprevisível; e o terceiro é a recompensa, que indica ao agente se seu comportamento foi bom ou ruim. A dinâmica completa pode ser descrita como um ciclo contínuo: o agente observa o estado do ambiente, escolhe uma ação, recebe uma recompensa e observa o novo estado resultante. A cada interação, ele atualiza seu conhecimento para melhorar seu comportamento futuro.

Um dos desafios mais importantes do aprendizado por reforço é o equilíbrio entre *exploration* e *exploitation*. *Exploration* significa tentar novas ações para descobrir comportamentos potencialmente melhores. *Exploitation*, por outro lado, significa utilizar ações cuja eficácia já é conhecida. Encontrar o equilíbrio entre essas duas estratégias é essencial para conseguir modelos eficientes e adaptativos, especialmente em ambientes complexos.

Exemplos reais ajudam a ilustrar o poder dessa abordagem. Em robótica, por exemplo, um braço robótico aprende a pegar objetos, abrir portas ou montar peças através de tentativa e erro. Em jogos, algoritmos de aprendizado por reforço foram responsáveis por conquistas históricas, como o AlphaGo derrotando campeões mundiais ou agentes que superam jogadores humanos em jogos complexos, como Dota 2 ou StarCraft. Em sistemas de recomendação, técnicas de reforço ajudam a personalizar conteúdos em tempo real, reagindo às escolhas e preferências do usuário.

A estrutura básica do aprendizado por reforço pode ser facilmente representada em um diagrama em que o agente e o ambiente formam um ciclo contínuo de decisões.

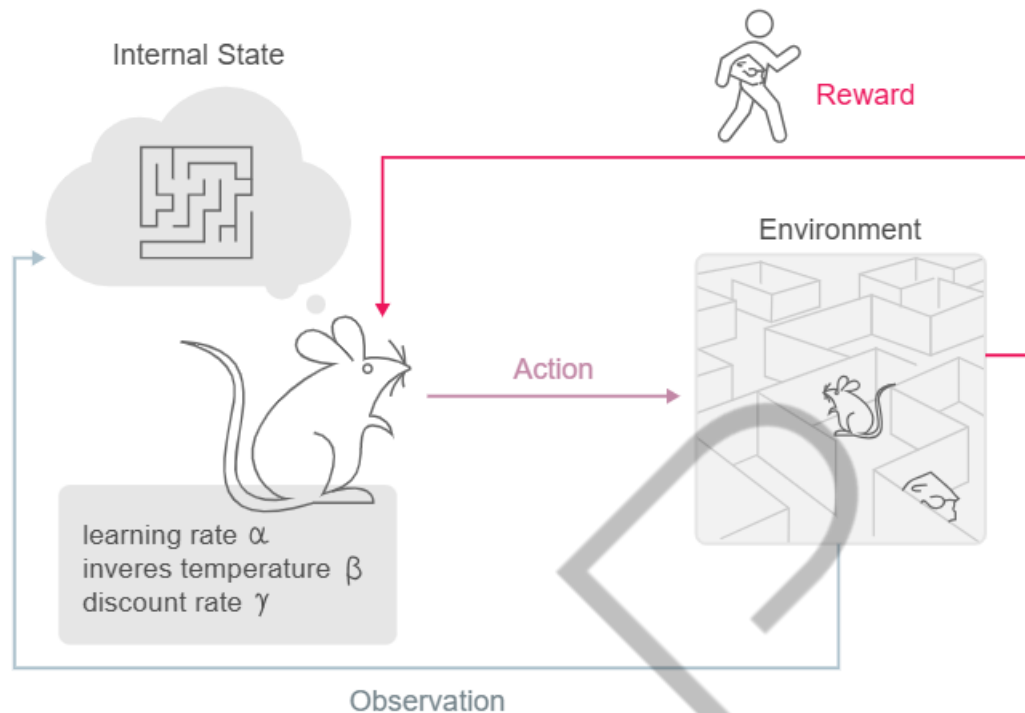


Figura 4 – Aprendizado por reforço
Fonte: GlassWing Ventures (2025)

2.4 Comparação entre os tipos de aprendizado

Agora que os três tipos principais de aprendizado foram apresentados, é possível compará-los sistematicamente. Embora todos compartilhem a ideia de aprender com dados, eles diferem radicalmente no tipo de informação disponível, na lógica de tomada de decisão e no tipo de problema que resolvem. O aprendizado supervisionado exige rótulos e está associado a previsões explícitas. O aprendizado não supervisionado procura padrões ocultos e estruturas internas. O aprendizado por reforço, por sua vez, aprende estratégias que maximizam recompensas ao longo do tempo.

Essas diferenças fazem com que cada abordagem seja mais apropriada para um conjunto distinto de situações. O supervisionado é ideal quando o objetivo é prever algo bem definido, como preços, classes, demandas ou diagnósticos. O não supervisionado é o mais adequado quando o objetivo é entender a estrutura dos dados, identificar grupos naturais ou detectar padrões incomuns. O por reforço é a melhor escolha quando é necessário aprender comportamentos sequenciais ou estratégias que evoluem com o tempo, especialmente em ambientes dinâmicos.

A escolha da abordagem correta depende, portanto, da natureza do problema e da disponibilidade de dados. Em muitos sistemas modernos, especialmente aqueles que combinam reconhecimento de padrões e tomada de decisão, as três abordagens coexistem e se complementam.

3 CLASSIFICAÇÃO, REGRESSÃO E CLUSTERING

O campo do aprendizado de máquina é movido pela ideia de identificar padrões e utilizá-los para fazer previsões ou entender melhor os dados. Para que isso aconteça de maneira estruturada, diferentes problemas são classificados em conjuntos específicos conforme o tipo de saída desejada. Entre essas categorias, três se destacam como as mais fundamentais: regressão, classificação e *clustering*. Essas três abordagens não apenas representam a maior parte dos problemas reais enfrentados por cientistas de dados e engenheiros de IA, mas também constituem a base sobre a qual arquiteturas modernas mais complexas, como redes neurais profundas e modelos generativos, são construídas.

Aqui, discutiremos cada um desses paradigmas em profundidade, explorando conceitos, aplicações práticas, intuições matemáticas, dificuldades comuns e exemplos ilustrativos. Também veremos como esses métodos se relacionam com a qualidade dos dados, a preparação das variáveis e a prevenção de erros como *overfitting* e *data leakage*.

3.1 Regressão

A regressão é o processo de prever valores numéricos contínuos a partir de variáveis de entrada. Em termos simplificados, consiste em estimar uma função que mapeia entradas para números reais. Problemas de regressão estão presentes em praticamente todos os setores da economia. Na indústria financeira, por exemplo, modelos de regressão são usados para estimar preços futuros de ativos, custos operacionais e valores de risco. No varejo, eles permitem prever vendas, demanda ou o efeito de promoções em produtos específicos. Em logística, ajudam a estimar o tempo de entrega, a necessidade de reposição e o custo de transporte.

A forma mais tradicional de regressão é a linear. Ela assume que existe uma relação linear entre as variáveis de entrada e a variável-alvo. Essa relação pode ser escrita como:

$$\hat{y} = w_1x_1 + \dots + w_nx_n + b$$

Nesse modelo, cada w representa um peso atribuído a uma variável de entrada, enquanto b é um termo de intercepto. O objetivo do processo de treinamento é ajustar esses valores de modo que a predição $\hat{y}$ se aproxime ao máximo do valor real y . Isso é feito minimizando uma função de perda, tipicamente o erro quadrático médio:

$$MSE = \left(\frac{1}{n}\right) \sum (y_i - \hat{y}_i)^2$$

A regressão linear é poderosa, mas não é adequada para situações em que as relações entre variáveis são complexas ou não lineares. Para esses casos, modelos mais sofisticados como regressão polinomial, regressão com kernels, árvores de decisão, *random forests* e redes neurais são empregados. Eles conseguem capturar interações e padrões mais sutis, especialmente quando há alto volume de dados.

Um exemplo prático ajuda a visualizar como a regressão funciona. Imagine que uma rede de supermercados deseja prever quantas unidades de determinado produto serão vendidas amanhã. As entradas podem incluir preço, promoções vigentes, dia da semana, clima, histórico de vendas e datas comemorativas. A regressão produzirá uma estimativa numérica da quantidade esperada de vendas, permitindo que gestores tomem decisões de compra mais eficientes. Em aplicações médicas, a regressão pode ser utilizada para prever a progressão de uma doença, como o nível futuro de glicemia de um paciente ou a evolução de marcadores clínicos ao longo do tempo.

A regressão é também uma ferramenta exploratória. Mesmo que o objetivo não seja necessariamente prever valores, mas entender como eles se relacionam com variáveis específicas, modelos de regressão ajudam a medir a força e a direção dessas relações, oferecendo informações valiosas para análises de causa e efeito.



Figura 5 – Regressão linear
Fonte: Cloudera Ventures (2019)

3.2 Classificação

A classificação é o processo de atribuir rótulos a entradas com base em características aprendidas durante o treinamento. Em vez de prever um valor numérico contínuo, como na regressão, a classificação prevê categorias discretas. Essa abordagem é amplamente usada para identificar padrões que distinguem elementos entre si.

Na prática, problemas de classificação aparecem em inúmeros contextos. Em segurança digital, sistemas classificam transações como fraudulentas ou legítimas. Em medicina, algoritmos classificam imagens como normais ou patológicas. Na indústria de varejo, modelos classificam clientes em categorias como *churn* iminente ou cliente fiel. Em linguagem natural, classificadores detectam o sentimento de textos, identificando se uma frase é positiva, neutra ou negativa.

A classificação binária é o tipo mais simples, envolvendo apenas duas categorias. Um exemplo clássico é o e-mail ser classificado como spam ou não spam. Já a classificação multiclasse envolve três ou mais categorias. O reconhecimento de espécies de flores usando o clássico conjunto Iris, por exemplo, é uma tarefa multiclasse. Existe ainda a classificação multilabel, em que um único item pode

pertencer a várias categorias simultaneamente, como na identificação de múltiplos objetos em uma imagem.

Para realizar essas tarefas, diversos algoritmos podem ser utilizados. A regressão logística, apesar do nome, é um dos classificadores mais simples e eficientes. Ela utiliza uma função sigmoide para transformar uma combinação linear das variáveis de entrada em uma probabilidade. Outras técnicas comuns incluem máquinas de vetor de suporte (SVM), árvores de decisão, redes neurais artificiais e métodos de *ensemble* como *random forests* e *gradient boosting*.

Um conceito essencial em classificação é o de fronteira de decisão, que representa a linha ou superfície que separa as classes no espaço de atributos. Quanto mais complexa for a relação entre as variáveis e as classes, mais complexa precisará ser essa fronteira. Por isso, modelos como redes neurais profundas conseguem capturar relações extremamente não lineares, sendo muito eficientes em tarefas como visão computacional.

Em problemas de classificação, as métricas utilizadas para avaliar desempenho são diferentes das métricas de regressão. Acurácia, precisão, *recall* e *F1-score* são comumente usados para medir a qualidade do modelo, especialmente em cenários com classes desbalanceadas. A área sob a curva ROC (AUC) também é frequentemente utilizada para avaliar quão bem o modelo separa as classes.

Para ilustrar, considere o exemplo de um modelo médico treinado para identificar pneumonia em radiografias de tórax. A amostra recebe como entrada uma imagem, realiza extração de características e produz uma probabilidade de que o paciente tenha a doença. A decisão final é tomada comparando essa probabilidade com um limiar. Esse processo exige enorme precisão, pois um falso negativo pode atrasar um diagnóstico importante, enquanto um falso positivo pode gerar ansiedade e exames desnecessários.

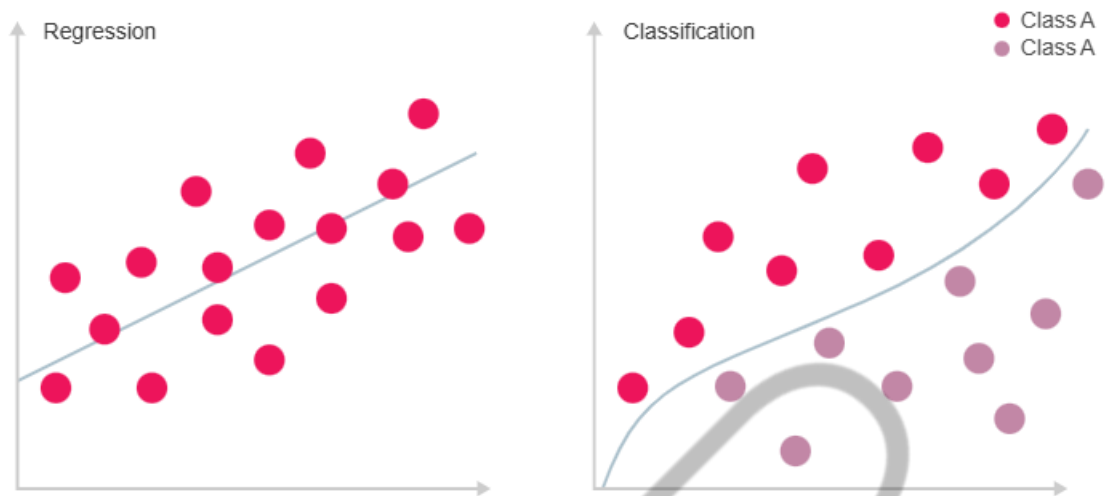


Figura 6 – Classificação versus regressão linear
Fonte: Pi.exchange (2023)

3.3 Clustering

Clustering, também conhecido como agrupamento, é uma técnica típica do aprendizado não supervisionado. Diferente da regressão e da classificação, aqui não existem rótulos previamente definidos. O objetivo é descobrir estruturas naturais nos dados, agrupando elementos que sejam semelhantes entre si e diferentes dos demais. Esses agrupamentos revelam padrões que não são imediatamente evidentes à primeira vista.

Um dos algoritmos mais conhecidos de *clustering* é o *K-Means*. Ele funciona selecionando um número pré-definido de grupos e garantindo que cada ponto seja atribuído ao centroide mais próximo. O algoritmo funciona de maneira iterativa, recalculando tanto as atribuições dos pontos quanto as posições dos centroides até chegar a uma configuração estável. Esse método é simples, eficiente e funciona bem quando os grupos têm formato aproximadamente esférico e variâncias semelhantes.

Outro algoritmo importante é o DBSCAN, que identifica agrupamentos com base na densidade dos pontos. Nesse caso, *clusters* são formados onde há grande concentração de elementos, enquanto pontos isolados são considerados ruídos ou anomalias. Isso torna o DBSCAN adequado para detectar padrões complexos, especialmente quando os grupos possuem formas irregulares.

O *clustering* possui aplicações amplas. Em marketing, é usado para segmentar clientes com base em comportamento de compra. Em sistemas de recomendação,

ajuda a identificar grupos de usuários com gostos semelhantes. Em detecção de anomalias, identifica padrões incomuns que podem indicar fraude, falha mecânica ou ataques cibernéticos. Em ciências biológicas, é usado para agrupar células, genes ou espécies com base em características morfológicas ou genéticas.

É importante destacar que, como não existem rótulos, avaliar a qualidade de um agrupamento é mais desafiador do que avaliar um classificador. Métricas como inércia e *silhouette score* ajudam a quantificar o quão bem os dados foram agrupados, medindo tanto a coesão interna quanto a separação entre *clusters*.

Imagine que uma empresa deseja entender melhor o comportamento de seus clientes. Ao analisar variáveis como frequência de visitas, gasto médio, variedade de produtos comprados e tempo entre compras, o algoritmo pode identificar grupos naturais como compradores esporádicos, clientes altamente fiéis e consumidores focados em promoções. Esses grupos podem não ser visíveis antes do *clustering*, mas, uma vez descobertos, permitem estratégias mais eficientes de marketing, fidelização e operação.

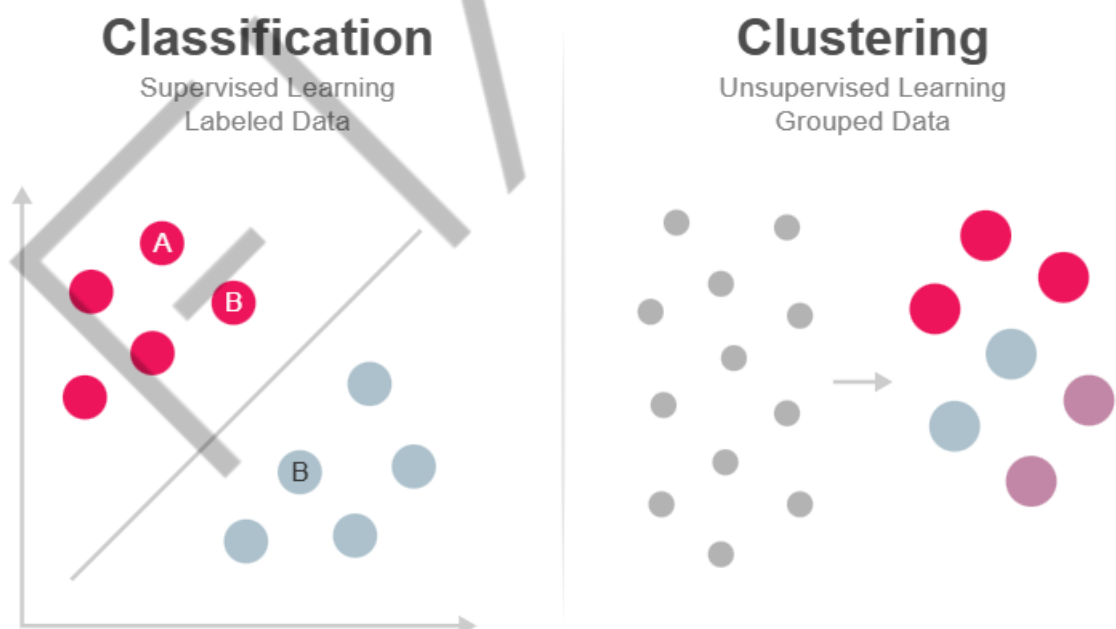


Figura 7 – *Clustering versus* classificação
Fonte: Vivek Vardhan (2025)

3.4 Considerações práticas

Independentemente de qual método esteja sendo utilizado, algumas considerações práticas são essenciais para garantir resultados confiáveis. A primeira

delas é o tratamento adequado dos dados. Em muitos casos, variáveis precisam ser normalizadas ou padronizadas para evitar que atributos de grande magnitude dominem o aprendizado do modelo. Isso é especialmente relevante em algoritmos baseados em distâncias, como *K-Means* ou SVM.

Outro ponto crítico é evitar o *data leakage*, que ocorre quando informações que não deveriam estar disponíveis durante o treinamento acabam influenciando o modelo. Esse erro pode causar resultados artificialmente elevados no conjunto de validação, levando a uma falsa sensação de desempenho. Para evitá-lo, é fundamental dividir corretamente os dados e aplicar transformações como escalonamento, imputação e *feature engineering* apenas após a separação entre treino e teste.

O fenômeno de *overfitting* também deve ser considerado. Ele ocorre quando o modelo aprende não apenas os padrões verdadeiros dos dados, mas também os ruídos e flutuações aleatórias. Como resultado, o desempenho no conjunto de teste ou em novos dados tende a ser fraco. Técnicas como regularização, validação cruzada, *early stopping* e aumento de dados ajudam a combater esse problema.

Além disso, o processo de *feature engineering*, criação, transformação e seleção de variáveis desempenha um papel central na qualidade do modelo. Muitas vezes, modelos simples com boas *features* superam modelos complexos com *features* inadequadas. A habilidade de construir e interpretar boas variáveis é um dos diferenciais de cientistas de dados experientes.

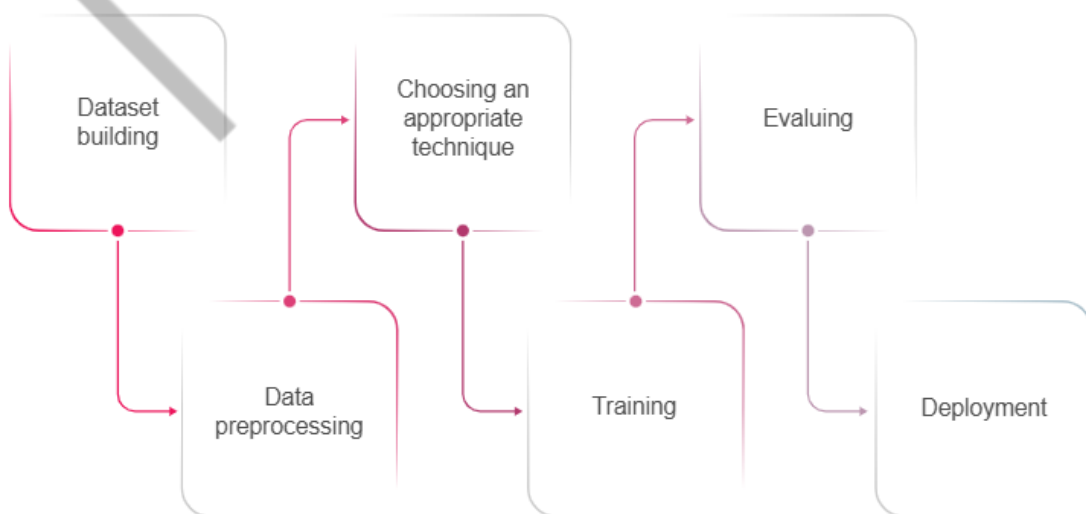


Figura 8 – Pipeline de projetos de machine learning
Fonte: ResearchGate (2021)

4 VALIDAÇÃO E AVALIAÇÃO DE MODELOS

A construção de um modelo de aprendizado de máquina não termina no momento em que o algoritmo é treinado. Pelo contrário, esse é apenas o início de um processo mais complexo: o de validar, avaliar e comparar modelos, garantindo que eles funcionem adequadamente em dados reais e desconhecidos. Um dos maiores erros que iniciantes cometem é acreditar que um modelo com excelente desempenho no conjunto de treinamento será igualmente bom em produção. A prática mostra que isso raramente é verdade. Sem uma estratégia sólida de validação, é impossível saber se o modelo está realmente aprendendo padrões relevantes ou apenas memorizando informações específicas daquele conjunto de dados.

Validar significa testar o modelo em dados que ele nunca viu, avaliar significa medir, com métricas apropriadas, seu desempenho nesses dados e comparar significa analisar diferentes modelos, arquiteturas e configurações com o objetivo de escolher a melhor alternativa possível. Aqui, exploraremos esses três pilares fundamentais, mostrando como cientistas de dados estruturam seus experimentos de maneira reproduzível e confiável.

4.1 Divisão de dados

A validação de modelos começa com a divisão adequada dos dados. O objetivo é separar os dados originais em subconjuntos que permitam treinar o modelo e testá-lo de maneira justa.

A divisão mais comum consiste na separação em três partes: treino, validação e teste. O conjunto de treino é utilizado para ajustar os parâmetros do modelo. O conjunto de validação serve para comparar modelos, ajustar hiperparâmetros e decidir qual configuração funciona melhor. Por fim, o conjunto de teste representa o julgamento final: ele só deve ser utilizado após todas as decisões terem sido tomadas, funcionando como uma simulação do desempenho real do modelo quando colocado em produção.

O conjunto de teste deve ser tratado como um recurso precioso, utilizado apenas no final. Se for consultado muitas vezes durante o desenvolvimento, perde sua função e o processo corre o risco de gerar *overfitting* ao próprio conjunto de teste.

Outra estratégia essencial é a validação cruzada, conhecida como *cross-validation*. Nesse esquema, os dados são divididos em k partes aproximadamente iguais. Em cada rodada, uma das partes é usada como validação e as outras $k - 1$ são usadas para treinamento. Esse processo é repetido k vezes, com cada parte sendo utilizada como validação uma vez. Ao final, calcula-se a média dos resultados. A validação cruzada é especialmente útil quando o conjunto de dados é pequeno ou quando se deseja obter uma estimativa mais estável do desempenho do modelo.

No entanto, a validação cruzada tradicional não deve ser utilizada quando os dados possuem dependência temporal, como séries históricas de vendas, preços ou sensores. Nesse caso, utilizar validação aleatória pode gerar vazamento temporal, permitindo que o modelo veja eventos futuros durante o treinamento. Para resolver isso, utiliza-se o *time series split* (ou divisão própria para séries temporais). Essa técnica cria divisões progressivas respeitando a ordem temporal. O modelo é treinado no passado e avaliado no futuro, repetidamente, de forma estruturada.

A validação para séries históricas é crucial em áreas como previsão de demanda, tendência financeira e detecção de anomalias industriais. Em qualquer situação em que o tempo influencia o comportamento do sistema, respeitar a ordem temporal garante que o modelo seja avaliado da forma correta.

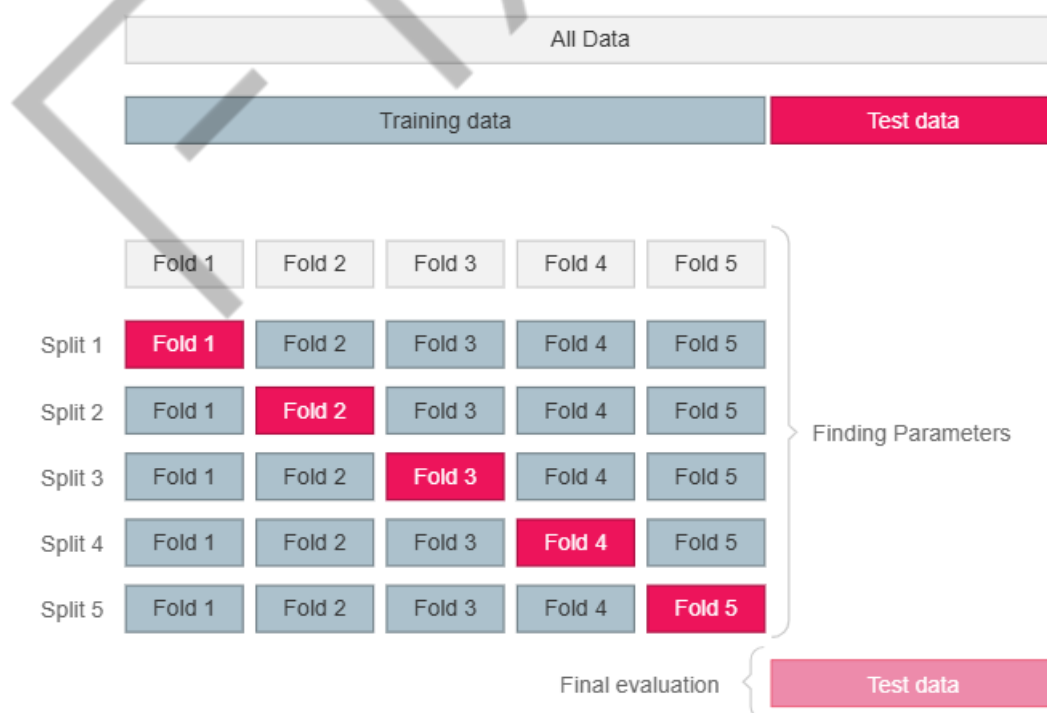


Figura 9 – Validação cruzada *K-fold*
Fonte: Scikit-Learn (2025)

4.2 Métricas de regressão

A avaliação de modelos de regressão exige métricas específicas que quantifiquem o erro entre os valores previstos e os valores reais. Entre as mais utilizadas estão o MSE, o RMSE, o MAE e o coeficiente de determinação R^2 .

O MSE, ou erro quadrático médio, penaliza erros proporcionais ao quadrado da sua magnitude. Ele é definido como:

$$MSE = \left(\frac{1}{n}\right) \sum (y_i - \hat{y}_i)^2$$

Por elevar o erro ao quadrado, o MSE oferece maior peso a grandes desvios. Essa característica pode ser desejável quando erros grandes representam situações mais prejudiciais.

O RMSE, ou raiz do erro quadrático médio, é simplesmente a raiz quadrada do MSE. Ele possui a vantagem de estar na mesma unidade da variável prevista, o que facilita a interpretação. Em previsão de vendas, por exemplo, um RMSE de 10 unidades significa que, em média, o modelo erra cerca de 10 unidades de produto.

O MAE, ou erro absoluto médio, calcula a média do valor absoluto das diferenças entre previsões e valores reais:

$$MAE = \left(\frac{1}{n}\right) \sum |y_i - \hat{y}_i|$$

O MAE não penaliza erros grandes de forma tão severa quanto o MSE; isso o torna mais robusto a *outliers*. Em muitas aplicações, especialmente quando a distribuição dos erros é assimétrica, o MAE captura melhor a qualidade das previsões.

Por fim, o coeficiente de determinação R^2 mede a proporção da variabilidade dos dados que é explicada pelo modelo. Um R^2 próximo de 1 indica que o modelo explica bem a variabilidade da variável-alvo. Um R^2 próximo de 0 sugere que o modelo não está aprendendo relações relevantes. Contudo, essa métrica não deve ser utilizada sozinha, pois pode ser enganosa em situações com alta multicolinearidade ou em modelos não lineares.

Em problemas reais, a escolha da métrica depende do impacto esperado dos erros. Em previsão de estoque, por exemplo, erros grandes são custosos, tornando

MSE e RMSE mais adequados. Em modelos que lidam com dados ruidosos, o MAE pode ser preferido por sua robustez.

4.3 Métricas de classificação

A avaliação de modelos de classificação requer um conjunto diferente de métricas, pois essas tarefas envolvem categorias discretas. A acurácia é a métrica mais simples e representa a proporção de previsões corretas. Embora seja intuitiva, ela pode ser enganosa em conjuntos desbalanceados. Por exemplo, em uma base com 99% de exemplos negativos e 1% de exemplos positivos, um modelo que simplesmente prevê sempre a classe negativa terá 99% de acurácia, mas será completamente inútil.

Por isso, métricas mais refinadas são utilizadas. A precisão mede a proporção de previsões positivas que realmente são positivas. Ela é útil quando se deseja evitar falsos positivos. O *recall*, por outro lado, mede quantos casos positivos foram efetivamente identificados pelo modelo, e é essencial quando falsos negativos são mais graves, como em diagnósticos médicos.

O *F1-score* combina precisão e *recall* em uma única métrica, representando a média harmônica entre elas. Essa métrica é especialmente desejável em problemas desbalanceados, pois exige que o modelo tenha bom desempenho simultaneamente em ambas as dimensões.

Outra métrica fundamental é a AUC-ROC, que mede a capacidade do modelo de separar as classes. A curva ROC representa o *trade-off* entre a taxa de verdadeiros positivos e a taxa de falsos positivos em diferentes limiares de decisão. A área sob essa curva (AUC) indica quão bem o modelo separa automaticamente as classes. Um modelo com AUC de 0,5 não possui capacidade discriminativa, enquanto um modelo com AUC próximo de 1 indica excelente separação.

A curva PR (*Precision-Recall*) também é importante, especialmente em cenários com forte desbalanceamento. Ela mostra como precisão e *recall* variam para diferentes limiares. Em muitos problemas reais, como identificação de transações fraudulentas, a curva PR é mais informativa do que a curva ROC.

A matriz de confusão é outra ferramenta essencial. Ela organiza o desempenho do modelo em uma tabela que mostra verdadeiros positivos, falsos positivos, verdadeiros negativos e falsos negativos, permitindo uma compreensão visual do tipo de erro mais frequente.

4.4 Métricas para *clustering*

Avaliar modelos de *clustering* é mais desafiador porque não existem rótulos verdadeiros para comparar diretamente. Por isso, utiliza-se métricas específicas que avaliam coesão e separação dos grupos.

A inércia é a soma das distâncias quadradas entre cada ponto e o centroide do *cluster* ao qual ele pertence. Quanto menor a inércia, mais compactos e coesos estão os *clusters*. Contudo, a inércia tende a diminuir conforme o número de *clusters* aumenta, tornando necessário utilizá-la juntamente com gráficos como o método do cotovelo, que identifica o ponto em que a redução da inércia deixa de ser significativa.

O *silhouette score* mede o quão bem cada ponto está inserido no seu *cluster*. Ele varia de -1 a 1. Valores próximos de 1 indicam que o ponto está bem agrupado. Valores próximos de 0 sugerem sobreposição entre *clusters* e valores negativos indicam que o ponto provavelmente foi alocado ao *cluster* errado. Essa métrica é amplamente usada porque leva em conta tanto a coesão interna quanto a separação entre *clusters*.

Essas métricas ajudam a decidir se o número de *clusters* escolhido é apropriado e se o algoritmo está capturando padrões verdadeiros nos dados.

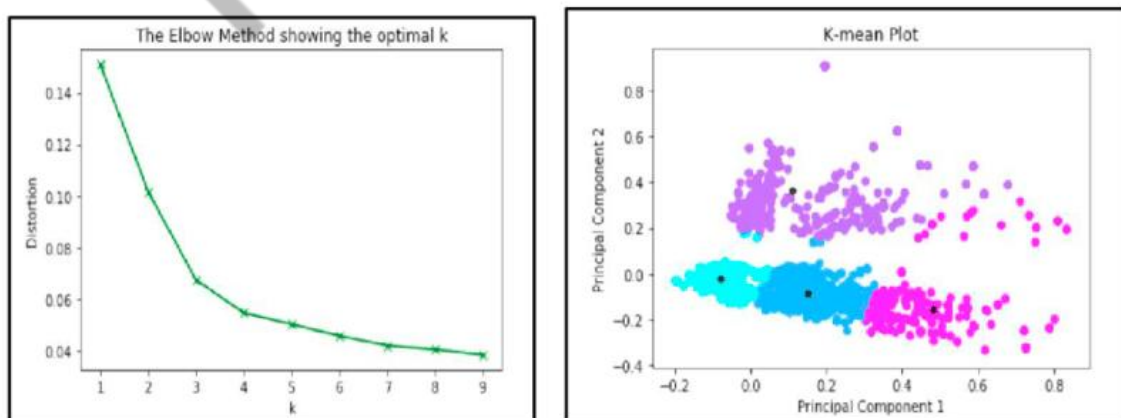


Figura 10 – Método do cotovelo
Fonte: ResearchGate (2025)

4.5 Comparando modelos e escolhendo o melhor

Depois que vários modelos foram treinados e avaliados, é necessário compará-los. Essa comparação deve considerar não apenas as métricas, mas também questões como interpretabilidade, custo computacional, robustez e capacidade de generalização.

Todo modelo apresenta um equilíbrio entre *bias* e *variance*. O *bias* representa a tendência do modelo de fazer suposições simplistas sobre os dados. Modelos com alto *bias* tendem a não aprender adequadamente, resultando em *underfitting*. Já a *variance* mede o quanto o modelo é sensível às variações nos dados de treino. Modelos com alta *variance* tendem a se ajustar demais, resultando em *overfitting*. O desafio é encontrar o equilíbrio ideal entre esses dois extremos.

Outro conceito importante é o teorema do não-almoço-grátis, que afirma que não existe um modelo universalmente superior para todos os problemas. O desempenho de um modelo depende da estrutura do problema e do tipo de dado disponível. Em outras palavras, a escolha só pode ser feita considerando o contexto específico de cada aplicação.

Comparar modelos implica analisar não apenas métricas numéricas, mas também sua estabilidade, comportamento diante de ruído, facilidade de implementação e compatibilidade com o ambiente de produção. Em muitos casos, modelos mais simples oferecem resultados mais robustos e interpretáveis, enquanto modelos mais complexos, como redes neurais profundas, podem capturar padrões muito mais sofisticados, desde que haja dados suficientes para treiná-los.

A escolha final deve ser guiada por uma combinação inteligente de experimentação, análise crítica e conhecimento do domínio da aplicação. A engenharia de IA não consiste apenas em treinar algoritmos, mas em selecionar, adaptar e validar modelos de forma responsável, eficiente e fundamentada.

5 REDES NEURAIS ARTIFICIAIS

As redes neurais artificiais são um dos pilares da inteligência artificial moderna. Elas se inspiram, de maneira altamente simplificada, no funcionamento do cérebro

humano, composto por bilhões de neurônios interconectados. Em vez de tentar reproduzir fielmente a biologia, as redes neurais adotam uma abstração matemática que captura a essência do aprendizado a partir de exemplos: ajustar pesos de conexões entre unidades simples para que, coletivamente, elas consigam representar funções altamente complexas.

Embora o sucesso recente de redes profundas seja frequentemente associado a grandes empresas de tecnologia, o conceito de neurônios artificiais remonta às décadas de 1940 e 1950. A partir do momento em que se percebeu que combinações de unidades simples poderiam construir funções sofisticadas, abriu-se um caminho para que a IA deixasse de ser apenas uma coleção de regras manuais e passasse a ser um sistema capaz de aprender com dados.

Aqui, vamos apresentar a intuição das redes neurais, discutir a arquitetura básica, explorar como o treinamento é realizado por meio de algoritmos de otimização, entender problemas como *vanishing* e *exploding gradients*, ver técnicas de regularização e, por fim, construir um exemplo prático em Python usando *Keras*.

5.1 Redes neurais: Intuição

O ponto de partida para entender redes neurais é o *perceptron*. Proposto por Frank Rosenblatt na década de 1950, esse conceito é um modelo extremamente simples de neurônio artificial. Ele recebe um conjunto de entradas, multiplica cada uma delas por um peso, soma esses produtos, adiciona um termo de viés (*bias*) e, em seguida, aplica uma função de ativação para produzir uma saída. Em termos matemáticos, podemos escrever:

$$z = w_1x_1 + \dots + w_nx_n + b$$

A saída do neurônio é dada por uma função de ativação aplicada a z . No *perceptron* clássico, essa função é do tipo degrau: se o valor de z for maior que um certo limiar, a saída é 1; caso contrário, é 0. Em problemas de classificação binária simples, o *perceptron* é capaz de separar dados que são linearmente separáveis, ou seja, que podem ser divididos por uma linha reta em duas regiões.

A grande limitação é justamente essa: ele só consegue resolver problemas em que uma fronteira de decisão linear é suficiente. Um exemplo famoso de problema

que o *perceptron* não consegue resolver é o XOR, em que a saída é 1 apenas quando as entradas são diferentes. Os pontos desse problema não podem ser separados por uma única linha reta no plano. Esse fato gerou, durante um período, um ceticismo em relação às redes neurais, até que se compreendeu que, ao empilhar vários neurônios em camadas, é possível superar essa limitação.

A intuição por trás das redes neurais modernas é a seguinte: um neurônio isolado é um classificador muito simples, capaz de traçar apenas uma fronteira linear. No entanto, quando combinamos muitos neurônios em camadas sucessivas, as saídas de uma camada servem como entradas para a próxima, e esse encadeamento permite construir fronteiras de decisão altamente não lineares. Em termos geométricos, cada camada aplica uma transformação no espaço de entrada, e as camadas profundas compõem diversas transformações, gerando representações cada vez mais sofisticadas.

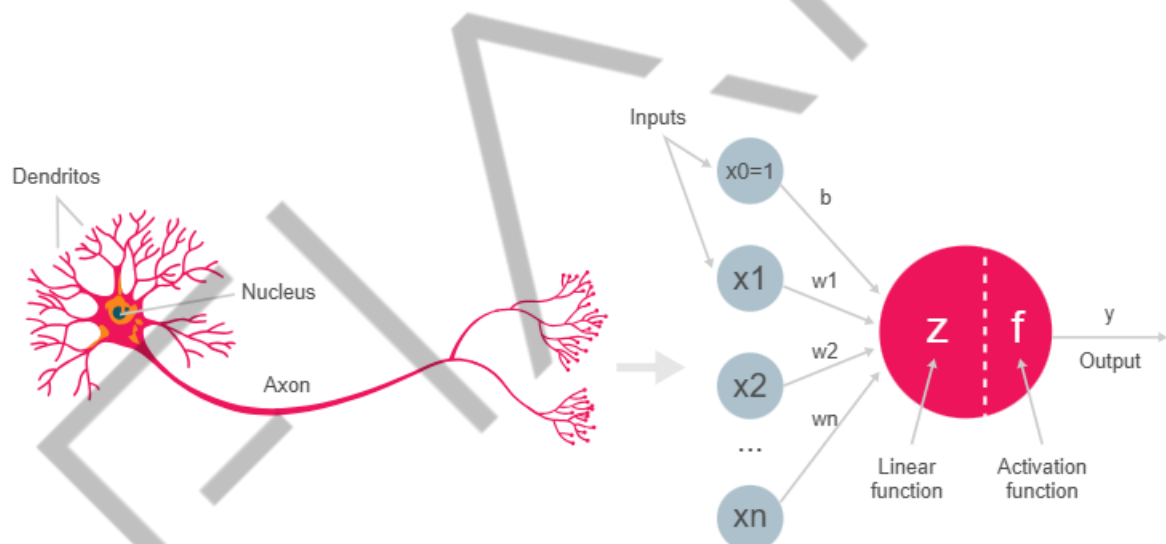


Figura 11 – *Perceptron*, neurônio artificial
Fonte: Ultralytics (2024)

5.2 Arquiteturas

As redes neurais artificiais mais simples são conhecidas como redes neurais *feedforward* ou *Perceptron* de Múltiplas Camadas (MLP). Nessas redes, a informação flui sempre em uma única direção: das entradas para as saídas, passando por camadas intermediárias chamadas de camadas ocultas. Não há ciclos ou conexões de retorno. Esse tipo de arquitetura é suficiente para resolver muitos problemas clássicos de classificação e regressão.

Uma rede *feedforward* típica começa com uma camada de entrada, que recebe os dados brutos, como vetores de características numéricas. Em seguida, vêm uma ou mais camadas densas, também chamadas de camadas totalmente conectadas. Em uma camada densa, cada neurônio recebe a entrada de todos os neurônios da camada anterior. A última, chamada de camada de saída, produz o resultado final do modelo, que pode ser um valor numérico, uma probabilidade ou um vetor de probabilidades, dependendo do tipo de problema.

Quando essas redes possuem muitas camadas ocultas são chamadas de *Deep Neural Networks* (DNNs). A profundidade da rede permite que ela aprenda representações hierárquicas dos dados. Por exemplo, em visão computacional, as primeiras camadas podem aprender bordas e contornos simples, as camadas intermediárias aprendem texturas e partes de objetos e, por fim, as camadas finais aprendem representações de alto nível, como categorias de objetos.

As funções de ativação desempenham um papel central no comportamento das redes. Sem elas, mesmo várias camadas empilhadas se comportariam, na prática, como uma única transformação linear. As funções de ativação introduzem não linearidade, permitindo que a rede represente funções complexas. As mais usadas incluem a *sigmoid*, a *tanh* e a *ReLU*. A função *sigmoid* transforma qualquer entrada em um valor entre 0 e 1, sendo muito usada em saídas de classificação binária. A *tanh* é semelhante, mas produz valores entre -1 e 1, o que pode facilitar o aprendizado em alguns cenários. A *ReLU*, ou unidade linear retificada, é definida como o máximo entre zero e a entrada. Ela é muito popular em redes profundas por ser simples e por ajudar a mitigar problemas de gradiente em muitas situações.

Cada uma dessas funções possui características específicas e escolhas diferentes podem impactar tanto a velocidade quanto a estabilidade do treinamento. Em redes modernas, é comum usar *ReLU* nas camadas internas e outras funções específicas, como *softmax*, na camada de saída, especialmente em problemas de classificação multiclasse.

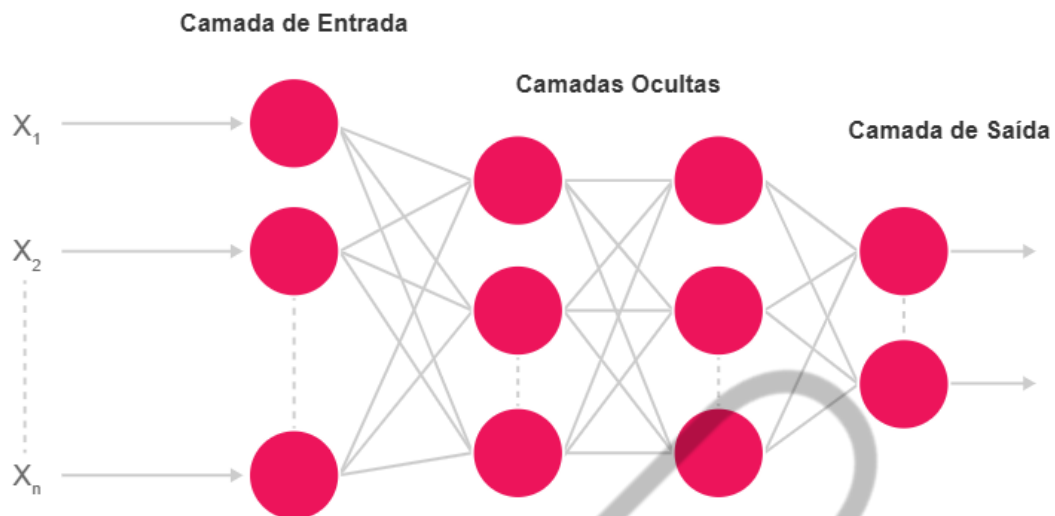


Figura 12 – Perceptron de múltiplas camadas
Fonte: Researchgate (2008)

5.3 Treinamento

O treinamento de redes neurais é, essencialmente, um problema de otimização. Dado um conjunto de treinamento composto por pares de entrada e saída, desejamos encontrar os valores dos pesos e vieses da rede que minimizam uma função de perda, como o erro quadrático médio em regressão ou a entropia cruzada em classificação.

O processo padrão utiliza alguma variante do gradiente descendente. A ideia central do gradiente descendente é ajustar os pesos na direção contrária ao gradiente da função de perda em relação a esses pesos. O gradiente indica a direção de maior aumento da perda. Portanto, caminhar na direção oposta reduz a perda. Em termos simples, cada passo de atualização de peso é dado por:

$$w = w - \eta \frac{\partial L}{\partial w}$$

η é a taxa de aprendizado, que controla o tamanho do passo, e $\partial L / \partial w$ é a derivada da função de perda em relação ao peso w .

O algoritmo que torna esse cálculo prático em redes neurais com muitas camadas é o *backpropagation*. Ele funciona aplicando a regra da cadeia da derivada para propagar o erro da camada de saída até as camadas mais internas. Na fase *forward*, a rede propaga as entradas, camada por camada, até gerar uma saída. Na fase *backward*, o erro é calculado na saída e propagado de volta, permitindo computar

as derivadas parciais de cada peso em relação à perda. Essas derivadas alimentam o gradiente descendente, que atualiza os pesos.

Embora o *backpropagation* possa parecer complexo à primeira vista, sua essência é bastante simples: ele é apenas um modo eficiente de aplicar cálculo diferencial em estruturas compostas por várias funções encadeadas. As bibliotecas modernas de *deep learning* automatizam esse processo através de grafos computacionais, deixando para o usuário apenas a tarefa de definir a arquitetura e escolher a função de perda e o otimizador.

O SGD, ou *Stochastic Gradient Descent*, é uma variação do gradiente descendente que, em vez de calcular o gradiente em todo o conjunto de dados, utiliza apenas um exemplo ou um pequeno lote de exemplos (*minibatch*) por vez. Isso torna o treinamento muito mais rápido em bases grandes e introduz um ruído benéfico ao processo de otimização, ajudando o modelo a escapar de mínimos locais.

Otimizadores mais avançados, como *Adam*, combinam ideias de *momentum* (velocidade de aumento ou diminuição da função de erro) e adaptação da taxa de aprendizado para cada parâmetro. O *Adam* mantém estimativas de primeira e segunda ordem dos gradientes e ajusta a taxa de aprendizado de forma adaptativa. Na prática, ele converge mais rapidamente em muitos problemas e é amplamente utilizado em redes profundas.

Do ponto de vista numérico, redes muito profundas podem enfrentar problemas de *vanishing* e *exploding gradients*. O *vanishing gradient* ocorre quando os gradientes se tornam extremamente pequenos durante a propagação para trás, fazendo com que camadas iniciais quase não aprendam nada. O *exploding gradient* é o problema oposto, em que os gradientes se tornam muito grandes, levando a atualizações instáveis e divergência do treinamento. Escolhas adequadas de funções de ativação, inicialização de pesos e arquiteturas específicas, como redes residuais, ajudam a mitigar esses problemas e tornam possível treinar redes com dezenas ou centenas de camadas.

5.4 Regularização

À medida que redes neurais crescem em tamanho e capacidade, torna-se cada vez mais importante controlar o *overfitting*, em que o modelo se ajusta tão bem aos

dados de treinamento que passa a memorizar ruídos e detalhes específicos, perdendo capacidade de generalizar para novos dados. A regularização é o conjunto de técnicas usado para combater esse problema.

Uma das técnicas mais populares é o *dropout*. Durante o treinamento, o *dropout* desativa aleatoriamente uma porcentagem dos neurônios em cada camada e em cada passo de atualização. Isso impede que a rede dependa demais de caminhos específicos e força o modelo a distribuir o aprendizado. Na prática, o *dropout* funciona como se diferentes sub-redes fossem treinadas em paralelo e, na fase de inferência, todas as conexões são usadas com pesos ajustados para compensar o efeito.

Outra técnica importante é o *early stopping*. Em muitos casos, o erro no conjunto de treinamento continua diminuindo à medida que o treinamento prossegue, mas o erro no conjunto de validação começa a aumentar após certo ponto. Isso é um sinal de que o modelo começou a sobreajustar os dados de treino. O *early stopping* interrompe o treinamento quando o desempenho no conjunto de validação deixa de melhorar, evitando *overfitting* e economizando recursos computacionais.

A normalização também é uma forma de regularização no sentido amplo. Métodos como *batch normalization* normalizam as ativações dentro das camadas, estabilizando a distribuição dos dados durante o treinamento. Isso facilita a otimização, permite o uso de taxas de aprendizado maiores e melhora a convergência. Além disso, o próprio ato de normalizar introduz um leve ruído estatístico durante o treinamento, o que também pode contribuir para reduzir o *overfitting*.

Técnicas adicionais, como L1 e L2 *regularization*, adicionam termos de penalização à função de perda, forçando os pesos a permanecerem pequenos ou até mesmo induzindo esparsidade. Em redes profundas, a combinação de várias estratégias de regularização é frequentemente necessária para obter modelos robustos e com boa capacidade de generalização.

6 DEEP LEARNING

O *deep learning* representa um avanço significativo dentro do campo de aprendizado de máquina e, atualmente, influencia praticamente toda aplicação moderna de inteligência artificial. Redes neurais profundas são capazes de aprender

representações extraordinariamente ricas a partir de dados brutos, dispensando boa parte do esforço manual de criação de *features* que caracterizava abordagens tradicionais. Da visão computacional ao processamento de linguagem natural, passando por sistemas de recomendação, robótica e modelos generativos, a maior parte das inovações recentes pode ser atribuída ao desenvolvimento e amadurecimento do *deep learning*.

Aqui, exploraremos o significado dessa profundidade, por que ela funciona tão bem em escala, quais arquiteturas modernas lideram o campo e os desafios práticos que acompanham essa tecnologia.

6.1 O que significa “profundo”?

O termo “*deep*” em *deep learning* refere-se à profundidade das redes, isto é, ao número de camadas ocultas presentes entre a entrada e a saída. Uma rede neural tradicional, com apenas uma ou duas camadas ocultas, pode aprender algumas relações não lineares, mas sua capacidade é limitada. Já uma rede profunda, composta por dezenas, centenas ou, em alguns casos, milhares de camadas, pode aprender funções extremamente complexas representando relações sofisticadas nos dados.

A profundidade favorece a construção de representações hierárquicas. Em vez de tentar aprender diretamente uma função complicada do espaço de entrada para o espaço de saída, a rede decompõe essa função em uma série de transformações simples. Cada camada aprende um nível de abstração maior que o anterior. Por exemplo, em visão computacional, uma camada inicial pode aprender a detectar bordas, a camada seguinte combina essas bordas em texturas e as camadas mais profundas reconhecem formas, partes de objetos e objetos completos. Esse processo de composição hierárquica é semelhante ao que ocorre no sistema visual humano, conforme comprovado por estudos neurocientíficos.

Essa capacidade de extrair representações cada vez mais ricas é o que torna o *deep learning* tão poderoso. Diferentemente de algoritmos tradicionais, que dependem de engenharia manual de características, redes profundas constroem automaticamente essas características, adaptando-se aos dados e ao problema.

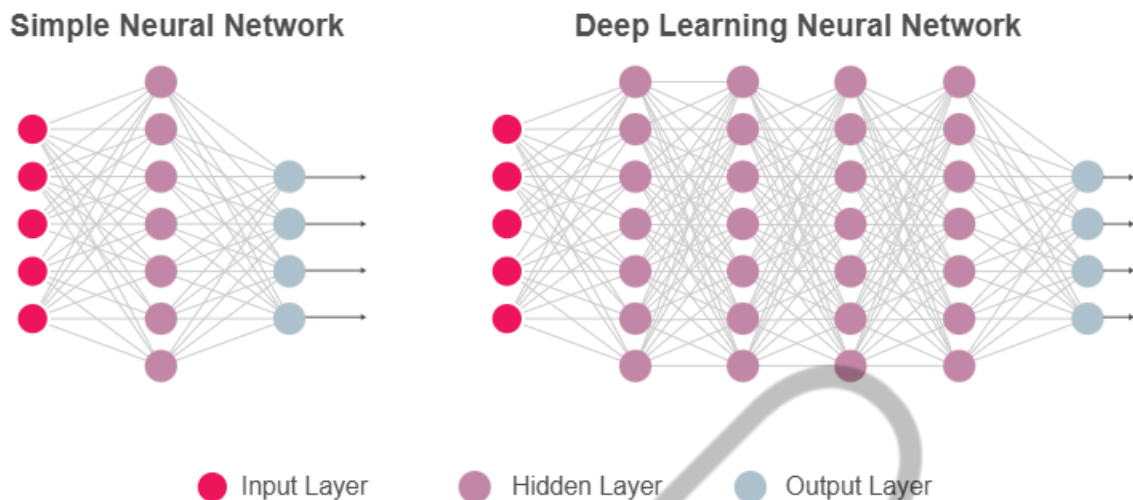


Figura 13 – MLP vs *deep learning*
Fonte: iMobilis UFOP (2019)

6.2 Por que *deep learning* funciona tão bem?

O sucesso do *deep learning* se deve a uma combinação de fatores tecnológicos, matemáticos e práticos. O primeiro deles é o *feature learning*, ou aprendizado automático de representações. Em métodos clássicos, como regressão logística, SVM ou *random forests*, o modelo opera diretamente sobre as *features* fornecidas pelo especialista. No *deep learning*, o modelo aprende suas próprias *features* a partir dos dados. Isso confere enorme flexibilidade e permite resolver problemas que seriam impossíveis com representações manuais.

O segundo fator é a disponibilidade de grandes volumes de dados. A ascensão da internet, redes sociais, sensores digitais, smartphones, transações financeiras e plataformas de mídia gerou um ambiente no qual trilhões de dados são produzidos diariamente. Redes neurais profundas prosperam quando expostas a grandes conjuntos de exemplos. Quanto mais dados, melhor o modelo tende a generalizar.

O terceiro fator é o avanço no poder computacional. O uso de GPUs e TPUs transformou o treinamento de redes profundas em algo viável. GPUs são extremamente eficientes para cálculos matriciais, que são a base das operações em redes neurais. Enquanto uma CPU tradicional processa algumas operações simultaneamente, uma GPU processa milhares, acelerando enormemente o aprendizado. As TPUs, desenvolvidas especificamente para *deep learning*, ampliaram ainda mais esse ganho.

Além disso, o conceito de grafo computacional, introduzido nas bibliotecas modernas de *deep learning*, permitiu automatizar o cálculo de gradientes. Isso possibilitou a construção de arquiteturas complexas sem a necessidade de derivar manualmente as funções, tornando o desenvolvimento de modelos acessível tanto para pesquisadores quanto para profissionais que não são especialistas em matemática avançada.

Essa combinação, *feature learning*, abundância de dados, poder computacional e grafos dinâmicos de cálculo, explica por que o *deep learning* dominou tantas áreas. Ele não é apenas uma evolução incremental dos métodos clássicos; é uma mudança radical de paradigma.

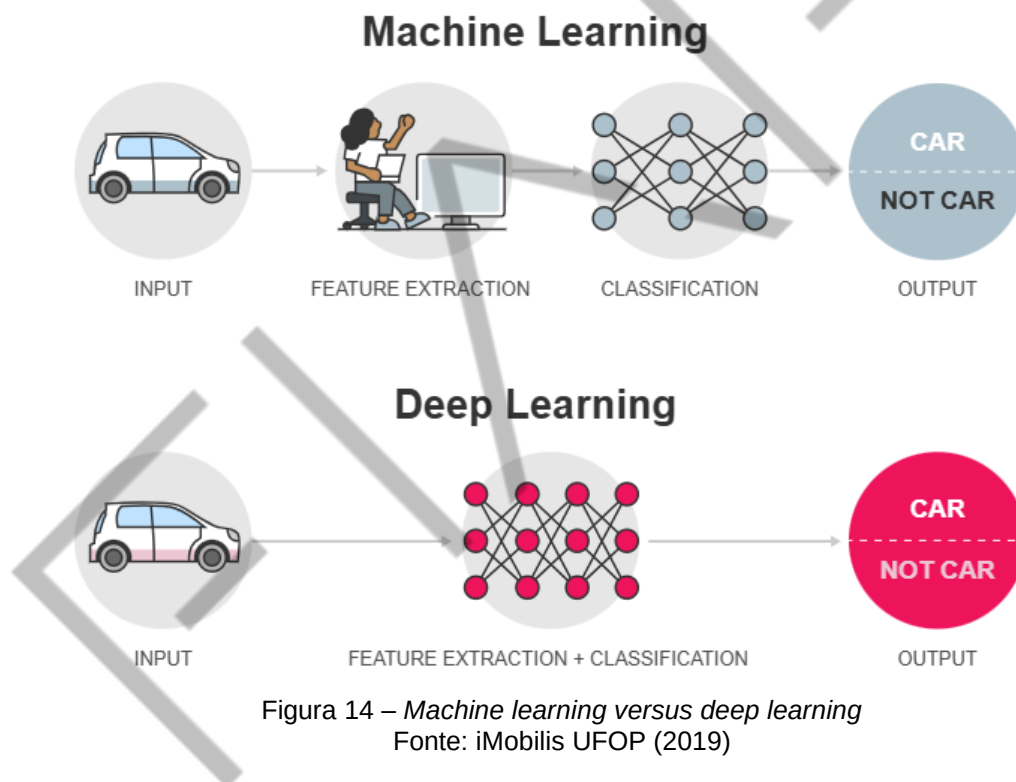


Figura 14 – Machine learning versus deep learning
Fonte: iMobilis UFOP (2019)

6.3 Arquiteturas

Existem diversas arquiteturas de *deep learning* projetadas para tipos específicos de dados e tarefas. Cada uma delas evoluiu ao longo de anos de pesquisa e otimização, tornando-se padrão em aplicações modernas.

As redes neurais convolucionais, ou CNNs, são amplamente utilizadas em visão computacional. Elas foram projetadas para explorar a estrutura espacial de imagens. Em vez de conectar cada pixel diretamente à próxima camada, as CNNs

utilizam filtros convolucionais que varrem a imagem, detectando padrões locais. Esses filtros são compartilhados por toda a imagem, reduzindo o número de parâmetros e tornando o modelo mais eficiente. Camadas convolucionais capturam bordas, texturas e padrões visuais, enquanto camadas posteriores agregam essas informações para detectar objetos, rostos ou cenas inteiras. As CNNs transformaram áreas como reconhecimento facial, diagnóstico médico por imagem, veículos autônomos e sistemas de vigilância.

As redes recorrentes, ou RNN's, foram criadas para lidar com dados sequenciais, como texto, áudio e séries temporais. Nelas, a saída de um neurônio em um instante é usada como entrada no instante seguinte, permitindo que a rede mantenha uma memória do passado. No entanto, RNNs simples sofrem com problemas de *vanishing* e *exploding gradients*, dificultando o aprendizado de dependências de longo prazo. Para resolver essas dificuldades, surgiram arquiteturas como LSTM e GRU, que adicionam mecanismos de portas capazes de controlar o fluxo de informação. Esses modelos foram fundamentais para avanços iniciais em tradução automática, análise de sentimentos e previsão de séries temporais.

Nos últimos anos, os *transformers* revolucionaram o campo de *deep learning*, especialmente no processamento de linguagem natural. Diferentemente das RNNs, os *transformers* não dependem de processamento sequencial. Eles utilizam mecanismos de atenção, que permitem que cada elemento da entrada interaja diretamente com todos os outros elementos. Isso reduziu drasticamente os tempos de treinamento e permitiu a construção de modelos gigantescos, como GPT, BERT e outros LLMs. O *transformer* não apenas domina NLP; ele também tem sido aplicado com sucesso em visão, áudio, *genomics* e modelos multimodais.

6.4 Desafios do *deep learning*

Apesar de seu enorme poder, o *deep learning* não está isento de desafios. Um dos maiores é o risco de *overfitting* extremo. Redes profundas possuem milhões ou até bilhões de parâmetros, o que as tornam altamente flexíveis, mas também vulneráveis a memorizar dados de treinamento. Sem técnicas adequadas de regularização, validação rigorosa e grande quantidade de dados, elas podem falhar ao generalizar.

Outro desafio importante é a interpretabilidade. Enquanto modelos lineares ou árvores de decisão oferecem explicações relativamente claras, redes profundas operam como “caixas pretas”. Sua lógica interna é distribuída por dezenas ou centenas de camadas, dificultando a compreensão do porquê uma decisão foi tomada. Em aplicações sensíveis, como saúde, justiça e finanças, essa falta de interpretabilidade pode ser problemática, motivando pesquisas em explicabilidade de modelos, como LIME e SHAP.

A necessidade de grandes volumes de dados também é um obstáculo. Modelos profundos exigem quantidades imensas de exemplos rotulados para alcançar um desempenho ótimo. Isso tornou o treinamento muito caro em certos domínios, como medicina, em que rotular dados exige especialistas altamente qualificados. Para mitigar esse problema, técnicas como auto-supervisão, augmentação de dados e *transfer learning* ganharam destaque.

Por fim, o custo computacional é um desafio significativo. Treinar modelos profundos exige hardware especializado, grande consumo energético e investimentos altos. Modelos de larga escala podem levar semanas para treinar, mesmo utilizando dezenas de GPU's. Além disso, implantar esses modelos em produção requer otimizações cuidadosas para garantir baixas latências e custos operacionais viáveis.

Mesmo com esses desafios, o *deep learning* continua evoluindo rapidamente. Novas arquiteturas, técnicas de compressão, modelos mais eficientes e avanços em hardware estão expandindo o alcance da tecnologia. Ao mesmo tempo, o entendimento desses desafios permite que engenheiros e cientistas de dados utilizem o *deep learning* de maneira mais consciente e estratégica, escolhendo a abordagem certa para cada problema.

7 LABORATÓRIO *HANDS ON*

Neste laboratório, analisaremos como treinar algoritmos para classificação e regressão no Python. Veremos também como utilizar algoritmos de clusterização e como treinar uma rede neural para reconhecer dígitos manuscritos.

7.1 Classificação

```
1 | from sklearn.datasets import load_iris
2 | from sklearn.model_selection import train_test_split
3 | from sklearn.preprocessing import StandardScaler
4 | from sklearn.linear_model import LogisticRegression
5 | from sklearn.metrics import accuracy_score
6 |
7 | # Carrega o dataset Iris
8 | data = load_iris()
9 | X = data.data
10 | y = data.target
11 |
12 | # Divisão treino/teste
13 | X_train, X_test, y_train, y_test = train_test_split(X,
14 | y,
15 | _size=0.2,
16 | om_state=42)
17 |
18 | # Normalização
19 | scaler = StandardScaler()
20 | X_train = scaler.fit_transform(X_train)
21 | X_test = scaler.transform(X_test)
22 |
23 | # Modelo simples: Regressão Logística
24 | clf = LogisticRegression(max_iter=500)
25 | clf.fit(X_train, y_train)
26 |
27 | # Avaliação
28 | y_pred = clf.predict(X_test)
29 | print("Acurácia (Classificação):",
30 | accuracy_score(y_test, y_pred))
```

Código-fonte 1 – Classificação em Python
Fonte: Elaborado pelo autor (2025)

7.2 Regressão

```
1 | from sklearn.datasets import fetch_california_housing
2 | from sklearn.model_selection import train_test_split
3 | from sklearn.linear_model import LinearRegression
4 | from sklearn.metrics import mean_squared_error
5 |
6 | # Carrega o dataset de preços de casas
7 | data = fetch_california_housing()
8 | X = data.data
```


O Surgimento da Inteligência que Apoia a Tomada de Decisão

```
9 | y = data.target
10|
11| # Divisão
12| X_train, X_test, y_train, y_test = train_test_split(X,
13| y,
14| _size=0.2,
15| om_state=42)
16| # Modelo de Regressão Linear
17| reg = LinearRegression()
18| reg.fit(X_train, y_train)
19|
20| # Predições e métricas
21| y_pred = reg.predict(X_test)
22| mse = mean_squared_error(y_test, y_pred)
23|
24| print("MSE (Regressão):", mse)
```

Código-fonte 2 – Regressão em Python
Fonte: Elaborado pelo autor (2025)

7.3 Clusterização

```
1 | from sklearn.datasets import make_blobs
2 | from sklearn.cluster import KMeans
3 |
4 | # Cria dados sintéticos para clusterização
5 | X, _ = make_blobs(n_samples=300, centers=3,
6 | random_state=42)
7 | # K-Means com 3 clusters
8 | kmeans = KMeans(n_clusters=3, random_state=42)
9 | kmeans.fit(X)
10|
11| print("Centroides dos clusters:")
12| print(kmeans.cluster_centers_)
13| print("Primeiras 10 previsões de cluster:",
14| kmeans.labels_[:10])
```

Código-fonte 3 – Clusterização em Python
Fonte: Elaborado pelo autor (2025)

7.4 Redes neurais

```
1 | import tensorflow as tf
2 | from tensorflow.keras.datasets import mnist
3 | from tensorflow.keras.models import Sequential
```

```
4 | from tensorflow.keras.layers import Dense, Flatten
5 | from tensorflow.keras.utils import to_categorical
6 |
7 | # Carrega MNIST
8 | (X_train, y_train), (X_test, y_test) = mnist.load_data()
9 |
10 | # Normalização
11 | X_train = X_train / 255.0
12 | X_test = X_test / 255.0
13 |
14 | # One-hot encoding
15 | y_train = to_categorical(y_train, 10)
16 | y_test = to_categorical(y_test, 10)
17 |
18 | # Modelo MLP simples
19 | model = Sequential([
20 |     Flatten(input_shape=(28, 28)),
21 |     Dense(128, activation='relu'),
22 |     Dense(10, activation='softmax')
23 | ])
24 |
25 | model.compile(optimizer='adam',
26 |               loss='categorical_crossentropy',
27 |               metrics=['accuracy'])
28 |
29 | # Treinamento (rápido)
30 | model.fit(X_train, y_train, epochs=3, batch_size=32,
31 |          verbose=0)
32 | # Avaliação
33 | loss, acc = model.evaluate(X_test, y_test, verbose=0)
34 | print("Acurácia da Rede Neural (MNIST):", acc)
```

Código-fonte 4 – Redes neurais em Python
Fonte: Elaborado pelo autor (2025)

8 CONCLUSÃO

Ao longo deste capítulo, percorremos os fundamentos essenciais da inteligência artificial moderna, construindo uma base sólida para todos os temas futuros. Exploramos a evolução histórica da IA, desde suas raízes conceituais com Turing até a revolução recente impulsionada pelo *deep learning*, compreendendo como avanços em dados, computação e algoritmos moldaram o cenário atual. Examinamos, em detalhes, os três grandes paradigmas de aprendizado de máquina:

supervisionado, não supervisionado e por reforço, identificando suas características, aplicações e limitações práticas.

Avançamos para os elementos centrais do aprendizado de máquina, analisando regressão, classificação e *clustering* como blocos fundamentais para qualquer sistema inteligente. Discutimos também a importância crucial da validação e avaliação de modelos, enfatizando que a construção de um sistema de IA confiável vai muito além do treinamento, exigindo rigor metodológico, métricas adequadas e entendimento dos *trade-offs* envolvidos.

Demos um passo adicional ao apresentar redes neurais artificiais e os princípios que sustentam as arquiteturas profundas, compreendendo como elas aprendem representações hierárquicas, por que funcionam tão bem em escala e quais desafios acompanham seu uso. Por fim, experimentamos, na prática, uma série de exemplos em Python, consolidando os conceitos mediante a construção de classificadores, modelos de regressão, algoritmos de *clustering* e uma pequena rede neural.

Esse conjunto de conhecimentos é a base para compreendermos sistemas ainda mais sofisticados, como modelos de linguagem, engenharia de prompts, RAG, arquiteturas multiagentes e integração de IA em sistemas complexos.

REFERÊNCIAS

JAMES, Gareth; WITTEN, Daniela; HASTIE, Trevor; TIBSHIRANI, Robert. *An Introduction to Statistical Learning: with Applications in R*. New York: Springer, 2013. (Springer Texts in Statistics, v. 103). ISBN 978-1-4614-7137-0.

RUSSELL, S., NORVIG, P. (2021). **Artificial Intelligence: A Modern Approach (4^a ed.)**. Pearson. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. Disponível em: <http://www.deeplearningbook.org>. Acesso em: 08 dez. 2025.

EMANIP